TP 2 — Nettoyage des données et analyse exploratoire (EDA)

Version étudiant

Établissement	ISIMM — Institut Supérieur d'Informatique et de Mathématiques de Monastir
Formation / niveau	3e année cycle ingénieur — Génie Logiciel
Module	Machine Learning — Chapitre 2 : Préparation des données et EDA
Durée	90 minutes
Modalité	Binôme — évaluation formative (non notée)
Jeu de données	Titanic (891 passagers, cible binaire `survived`)

1. Objectifs

À la fin de la séance, vous serez capables de :

- **O1** — Auditer un jeu de données : dimensions, types, valeurs manquantes.
- **O2** — Appliquer la règle anti-fuite : **séparer train/test avant tout calcul de statistique**.
- **O3** — Mener une EDA sur l'ensemble d'apprentissage et en interpréter les résultats.
- **O4** — Construire un pipeline de prétraitement avec `ColumnTransformer` : imputation, encodage, standardisation.
- **O5** — Vérifier un prétraitement par des assertions et diagnostiquer une fuite de données dans un code fourni.

2. Prérequis

- Lab 1 : manipulation de tableaux NumPy et de DataFrames Pandas (`groupby`, `isna`, indexation).
- Chapitre 2 : valeurs manquantes, encodage, mise à l'échelle, corrélation, **fuite de données** (*data leakage*).

3. Mise en situation

Une équipe vous confie un jeu de données historique pour préparer un futur modèle de classification. Avant toute modélisation, il faut livrer des données **propres, documentées et préparées sans fuite d'information**. Une préparation mal faite donne des scores d'évaluation trop optimistes, et cela passe souvent inaperçu. Votre livrable : un notebook qui audite, explore et prépare les données, avec des vérifications automatiques.

4. Préparation de l'environnement (Exercice 0 — 10 min)

Fichiers fournis dans le dossier du TP :

- `Lab02_starter.ipynb` : le notebook à compléter.
- `data/titanic.csv` : la copie locale du jeu de données, pour travailler hors ligne.

Étapes

1. Ouvrez un terminal **dans le dossier du TP**, celui qui contient `Lab02_starter.ipynb` et `data/`.
2. Vérifiez que Python 3.10 ou une version ultérieure est installé : `python --version`.
3. Installez les dépendances si elles manquent :

```
pip install numpy pandas matplotlib seaborn scikit-learn notebook
```

4. Lancez Jupyter : `jupyter notebook`, puis ouvrez `Lab02_starter.ipynb`.
5. Exécutez la première cellule. Elle doit afficher `(891, 8)`.

Solution de repli. Sans Internet, le code charge automatiquement `data/titanic.csv` via `sns.load_dataset("titanic", data_home="data")`. Si l'installation échoue, prévenez l'enseignant.

Activation des prérequis. Répondez par écrit, en deux minutes maximum :

- a. Que renvoie `df.isna().sum()` ?
- b. Quelle est la différence entre `fit` et `transform` pour un transformateur scikit-learn ?
- c. Définissez la fuite de données en une phrase.

5. Données

Variable	Type	Description
`survived`	entier 0/1	**Cible** : 1 = a survécu
`pclass`	entier 1/2/3	Classe du billet (1 = première)
`sex`	texte	`male` / `female`
`age`	réel (années)	Âge du passager
`sibsp`	entier	Nombre de frères, sœurs et conjoints à bord
`parch`	entier	Nombre de parents et enfants à bord
`fare`	réel	Prix du billet (unité monétaire d'époque, non convertie)
`embarked`	texte	Port d'embarquement : `C`, `Q`, `S`

Source. Jeu de données `titanic` distribué avec la bibliothèque seaborn. Il s'agit de données historiques réelles, pas de données synthétiques.

Limites. Ces données reflètent des règles sociales de 1912 (priorité aux femmes et aux classes aisées). Elles servent à l'apprentissage. Elles ne modélisent aucune situation actuelle.

6. Exercices

> **Règle d'or du TP :** aucune statistique (médiane, moyenne, mode, écart-type, catégories) ne doit être calculée sur des données qui contiennent l'ensemble de test.

Exercice 1 — Audit du jeu de données (10 min) · O1

Prédiction (avant d'exécuter). Quelles colonnes contiennent, selon vous, des valeurs manquantes ? Dans quelle proportion ?

1. Affichez `df.shape`, `df.dtypes` et `df.head()`.
2. Calculez `missing = df.isna().sum()` et affichez-le.
3. Calculez le pourcentage de valeurs manquantes de `age`.
4. Calculez le taux de survie global.
5. Tracez un diagramme en barres de `missing`.

Livrable : cellule exécutée, puis réponse écrite : *quelles colonnes faudra-t-il imputer ?*

Point de contrôle : vous devez trouver deux colonnes incomplètes. Comparez avec votre prédiction.

Indices gradués

1. `isna()` renvoie un tableau de booléens. Que donne leur somme ?
2. Pourcentage = nombre de manquants / `len(df)` × 100.
3. Taux de survie = moyenne d'une variable 0/1.

Exercice 2 — Séparer d'abord (5 min) · O2

1. Complétez `train_test_split` avec `test_size=0.25`, `random_state=42` et `stratify=y`.
2. Affichez les dimensions, puis le taux de survie de `y_tr` et de `y_te`.
3. Ajoutez une assertion vérifiant que les index de train et de test sont disjoints.

Question 2.4. À quoi sert `stratify=y` ? Comparez les deux taux de survie obtenus.

Point de contrôle : train = 668 lignes, test = 223 lignes.

Indices gradués

1. La fonction renvoie quatre objets, dans l'ordre `X_tr, X_te, y_tr, y_te`.
2. `set(X_tr.index).isdisjoint(X_te.index)`.

Exercice 3 — Analyse exploratoire sur l'ensemble d'apprentissage (15 min) · O3

Travaillez **uniquement** sur `train = X_tr.assign(survived=y_tr)`.

1. Calculez le taux de survie par `sex` et par `pclass`, puis tracez deux diagrammes en barres.
2. Calculez la moyenne et la médiane de `fare` et de `age`, ainsi que l'asymétrie (*skewness*) de `fare` avec `.skew()`.
3. Calculez la matrice de corrélation de `age`, `fare`, `sibsp`, `parch` et `survived`, puis affichez-la en carte de chaleur avec `sns.heatmap`.
4. Tracez les histogrammes de `age` et de `fare`.

Questions d'interprétation

- **3.5** Quelles variables semblent les plus liées à la survie ? Justifiez avec vos chiffres.
- **3.6** Comparez la moyenne et la médiane de `fare`. Que révèle l'écart ? Si `fare` contenait des valeurs manquantes, quelle valeur d'imputation choisiriez-vous, et pourquoi ?
- **3.7** Pourquoi mène-t-on l'EDA sur l'ensemble d'apprentissage plutôt que sur le jeu complet ?

Indices gradués

1. `train.groupby("sex")["survived"].mean()`.
2. Une asymétrie très supérieure à 1 indique une longue queue à droite.
3. Pour 3.7, relisez la règle d'or du TP.

Exercice 4 — Pipeline de prétraitement sans fuite (20 min) · O4

Prédiction. Après transformation, combien de colonnes aura la matrice ? Détaillez votre calcul : 4 colonnes numériques, plus combien de colonnes pour chaque variable catégorielle ?

1. `numeric_pipe` : `SimpleImputer(strategy="median")` puis `StandardScaler()`.
2. `categorical_pipe` : `SimpleImputer(strategy="most_frequent")` puis `OneHotEncoder(handle_unknown="ignore")`.
3. `pre` : un `ColumnTransformer` qui envoie `numeric` vers le premier pipeline et `categorical` (`sex`, `embarked`,

`pclass`) vers le second, avec `sparse_threshold=0`.
4. Apprenez les statistiques sur l'ensemble d'apprentissage : `X_tr_prep = pre.fit_transform(X_tr)`.
5. Appliquez-les au test : `X_te_prep = pre.transform(X_te)`.
6. Affichez les dimensions et `pre.get_feature_names_out()`.

Questions

- **4.7** Pourquoi encoder `sex` et `embarked` en one-hot plutôt qu'avec des codes entiers (0, 1, 2) ?
- **4.8** Ici, `pclass` est aussi encodée en one-hot. Donnez un argument pour ce choix et un argument pour conserver `pclass` telle quelle (variable ordinale).
- **4.9** Contre quoi protège `handle_unknown="ignore"` ?

Point de contrôle : la matrice d'apprentissage et celle de test ont le même nombre de colonnes. Ce nombre correspond-il à votre prédiction ?

Indices gradués

1. `Pipeline([("imp", ...), ("sc", ...)])`.
2. `ColumnTransformer([("num", numeric_pipe, numeric), ("cat", categorical_pipe, categorical)], sparse_threshold=0)`.
3. On n'appelle **jamais** `fit` sur `X_te`.

Exercice 5 — Vérifier et diagnostiquer (10 min) · O5

5a — Vérifications automatiques

1. Ajoutez une assertion vérifiant qu'il ne reste aucun `NaN` (`np.isnan(...).any()`).
2. Ajoutez une assertion vérifiant que les quatre premières colonnes de `X_tr_prep` ont une moyenne d'environ 0 et un écart-type d'environ 1 (`np.allclose`).
3. Affichez les moyennes et écarts-types des quatre colonnes numériques de `X_te_prep`.

Prédiction 5.3. Les moyennes sur le test seront-elles exactement 0 ? Justifiez **avant** d'exécuter.

5b — Trouver le bug. Le notebook contient le code suivant :

```
scaler = StandardScaler()
fare_scaled = scaler.fit_transform(X[["fare"]])
fs_tr, fs_te = train_test_split(fare_scaled, test_size=0.25,
                                random_state=42, stratify=y)
```

- **5.4** Ce code s'exécute sans erreur. Où est le problème ? Répondez **avant** de l'exécuter.
- **5.5** Écrivez la version corrigée. Comparez la valeur de `scaler.mean_` dans les deux versions.

Indices gradués

1. Sur quelles lignes la moyenne de `fare` est-elle calculée dans ce code ?
2. Ces lignes contiennent-elles des passagers de test ?

7. Extensions facultatives (hors temps obligatoire)

- **E1 — Transformation logarithmique.** Appliquez `np.log1p` à `fare`, puis comparez l'asymétrie et l'histogramme avant et après.
- **E2 — Indicateur de valeur manquante.** Comparez le taux de survie des passagers dont l'âge est manquant avec celui des autres. Testez ensuite `SimpleImputer(strategy="median", add_indicator=True)`. Combien de colonnes obtenez-vous ?
- **E3 — Artefact d'imputation.** Combien d'âges sont imputés dans l'ensemble d'apprentissage ?

Tracez l'histogramme de `age` après imputation et commentez.

8. Synthèse à compléter

1. L'ordre correct des étapes est : audit → _____ → EDA (sur _____) → `fit` du pipeline sur _____ → `transform` sur _____.
2. La médiane est préférable à la moyenne pour imputer une variable _____ .
3. La standardisation centre et réduit, mais ne supprime pas _____ .
4. Une fuite de données rend l'évaluation _____ .

9. Vérification individuelle des acquis (5 min, sans notebook)

1. Laquelle des deux méthodes apprend des statistiques : `fit` ou `transform` ?
2. Expliquez le nombre de colonnes obtenu après le prétraitement.
3. Une variable standardisée sur l'ensemble d'apprentissage a une moyenne de 0,09 sur le test. Est-ce un bug ? Justifiez.
4. La standardisation corrige-t-elle l'asymétrie de `fare` ?
5. La forte différence de survie selon `sex` est-elle une opportunité de modélisation, un enjeu d'équité, ou les deux ? Argumentez en trois lignes.

10. Remise et critères de réussite

À remettre en fin de séance : le notebook `Lab02_<nom1>_<nom2>.ipynb`, exécuté de haut en bas sans erreur, avec les réponses écrites dans les cellules Markdown.

Critère	Acquis	En cours	Non acquis
Audit complet : manquants identifiés et chiffrés (O1)			
Séparation effectuée avant toute statistique, avec assertion (O2)			
EDA sur l'ensemble d'apprentissage, interprétation chiffrée (O3)			
`ColumnTransformer` correct, `fit` sur train uniquement (O4)			
Assertions passées et bug de fuite expliqué (O5)			
Notebook réexécutable de haut en bas			

Glossaire français – anglais

Français	English
valeur manquante	missing value
imputation	imputation
fuite de données	data leakage
ensemble d’apprentissage / de test	training / test set
séparation stratifiée	stratified split
encodage one-hot	one-hot encoding
standardisation	standardization
asymétrie	skewness
analyse exploratoire	exploratory data analysis (EDA)
catégorie inconnue	unknown category